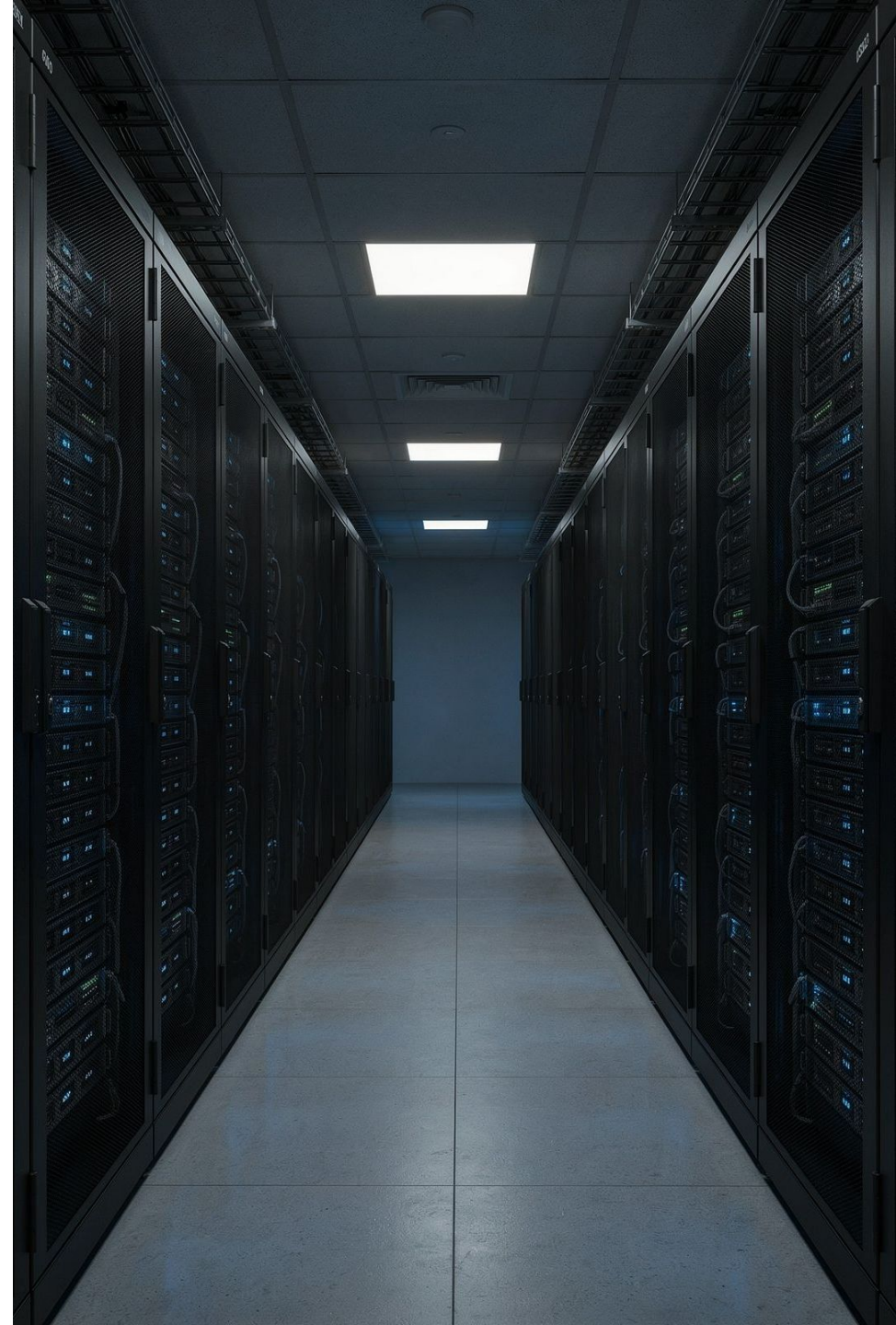


Day 4 — Credentials, Artifactory & Deployment

Securing secrets, managing artifacts, and controlling deployments in Jenkins CI/CD pipelines.



Learning Objectives

Credentials

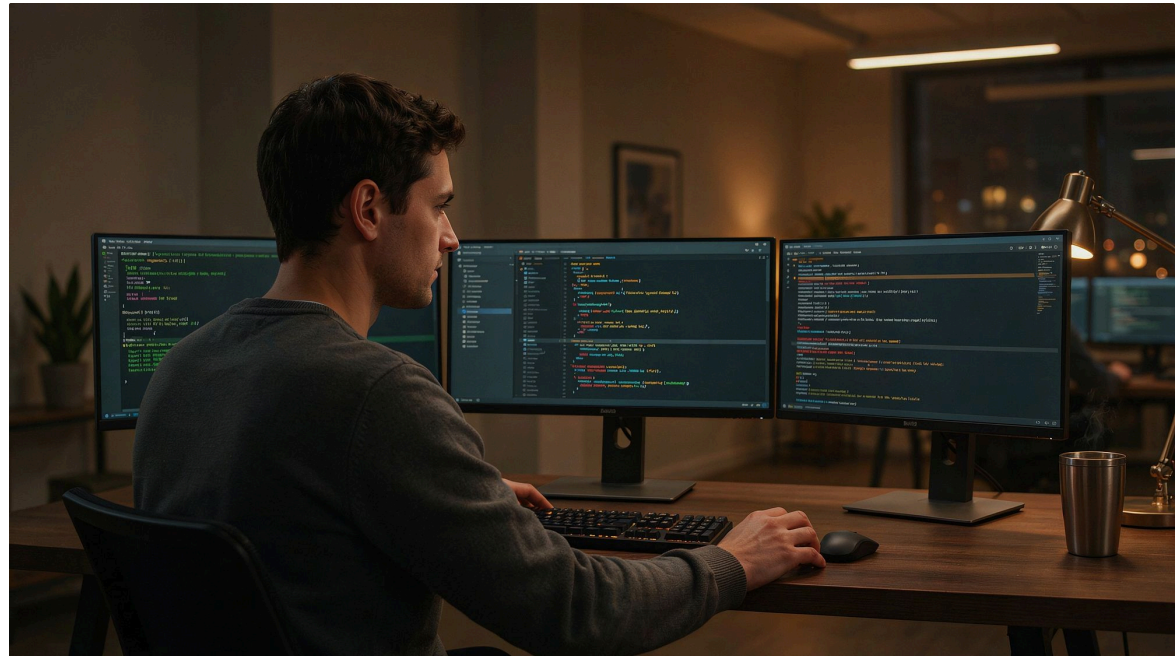
Understand the Jenkins Credentials Store, common types, and how to use secrets securely without exposing them in logs.

Artifact

Explain artifact repositories, JFrog repository types, and how to publish, retrieve, and promote build artifacts.

Deployment

Apply Build Once, Deploy Many, add manual approval gates, and understand basic rollback strategies.



📄 **Hands-on:** Secure Credentials → Build → Publish Artifact → Approval



SECTION 1

Jenkins Credentials

Why Do Pipelines Need Credentials?

CI/CD Pipelines communicate with many protected systems. The challenge: **how do we provide access without putting passwords or tokens directly in the Jenkinsfile?**



Git Repos

Private repository access



Artifact Repos

Publish and retrieve artifacts



Deploy Servers

SSH and remote access



Cloud & APIs

Databases, APIs, cloud services

Never Hardcode Secrets

Avoid

```
environment {  
  USERNAME = 'admin'  
  PASSWORD = 'Password123'  
}
```

Secrets may leak via source-control history, Jenkinsfile, code reviews, logs, screenshots, or backups.

Better Approach

Secret

**Jenkins
Credentials Store**

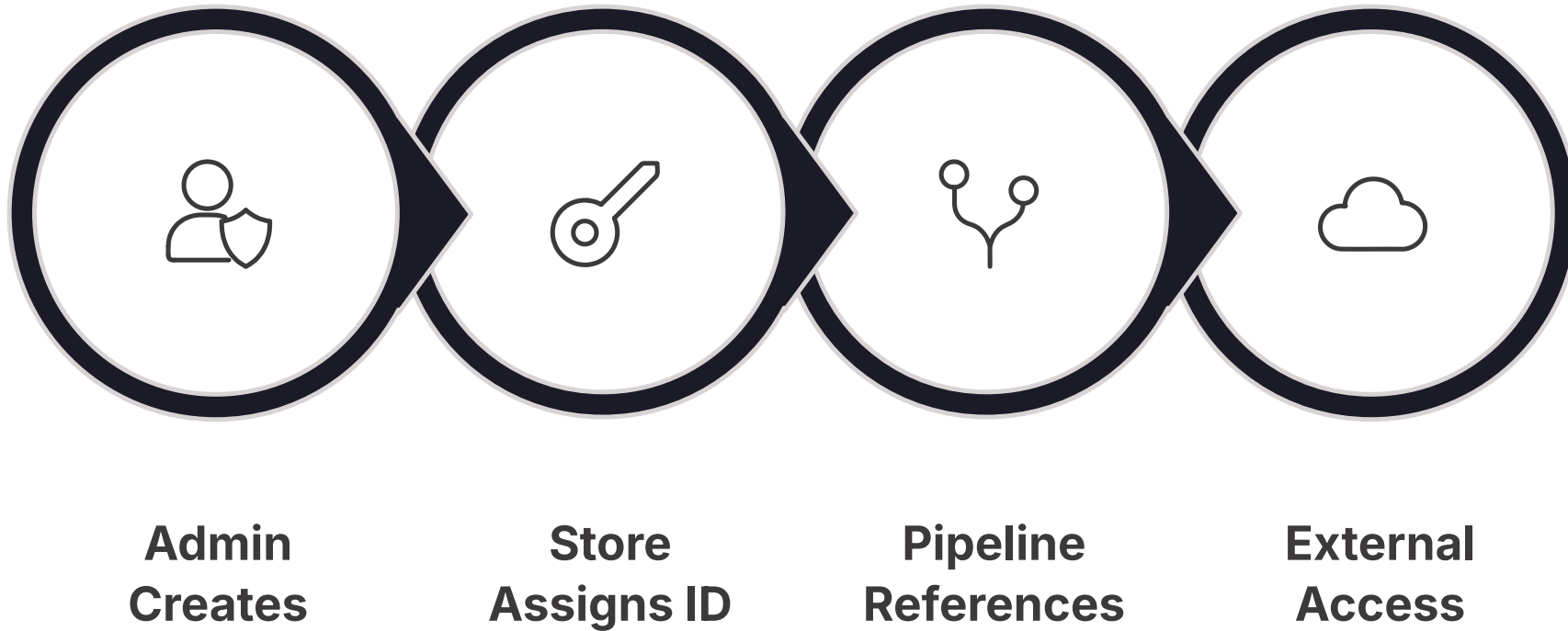
Pipeline

External System

Keep secrets outside application and Pipeline code.

Jenkins Credentials Store

Jenkins provides a **Credentials Store** for managing credentials required by jobs and Pipelines. The Pipeline refers to the credential using its configured **Credential ID** — never embedding the secret directly.



Common Credential Types



Username & Password

Used when a system requires both values — e.g. repository or artifact repository access.



Secret Text

Stores a single sensitive value such as an API token or access token.



SSH Key

Used for SSH-based authentication to remote systems via a private key.

Choose the credential type based on the authentication mechanism required by the external system.

Using Credentials in a Pipeline

```
stage('Connect') {  
  steps {  
    withCredentials([  
      usernamePassword(  
        credentialsId: 'repo-credentials',  
        usernameVariable: 'USER',  
        passwordVariable: 'PASS'  
      )  
    ]){  
      sh 'echo "Connecting..."'  
    }  
  }  
}
```

Important Principle

Use credentials **only for the steps that need them**. The `withCredentials` block creates a temporary pipeline context — secrets are never available outside it.

❏ Do not deliberately print secrets into Pipeline output.

Protect Secrets in Pipeline Logs

Avoid

```
sh 'echo $PASS'
```

Logs may be viewed by developers, administrators, support teams, and CI/CD operators.

Good Practices

- Do not echo passwords or tokens
- Do not hardcode secrets
- Limit credential scope
- Use meaningful credential IDs
- Expose credentials only where required

 **Key Principle:** Logs should help troubleshoot the Pipeline without exposing sensitive information.



SECTION 2

Artifact Management

Why Do We Need an Artifact Repository?

Teams need a central location to manage build outputs across the delivery process.



Artifact Repository Purpose

- Central artifact storage
- Controlled artifact retrieval
- Version management
- Build/deployment separation
- Artifact traceability



JFrog Artifactory Overview

Artifactory provides repositories for storing and managing software artifacts. The CI pipeline publishes `sample-app-1.0.105.jar` to Artifactory, where it can later be retrieved for deployment — without rebuilding.

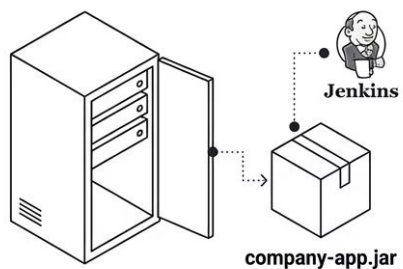
Git

Jenkins

Maven Build

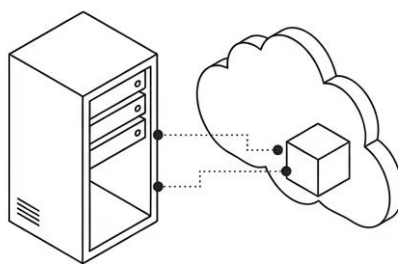
Application Artifact

Artifactory Repository Types



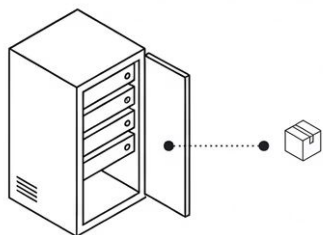
LOCAL REPOSITORY

Stores artifacts produced by your organization.



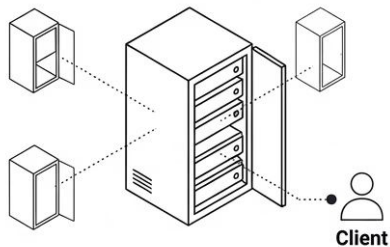
REMOTE REPOSITORY

Provides access to artifacts from an external repository through



VIRTUAL REPOSITORY

Provides a single logical access point combining Local, through Artifactory.



VIRTUAL REPOSITORY

Provides a single logical access point combining Local, Remote, and Other repositories.

Quick Reference

Local

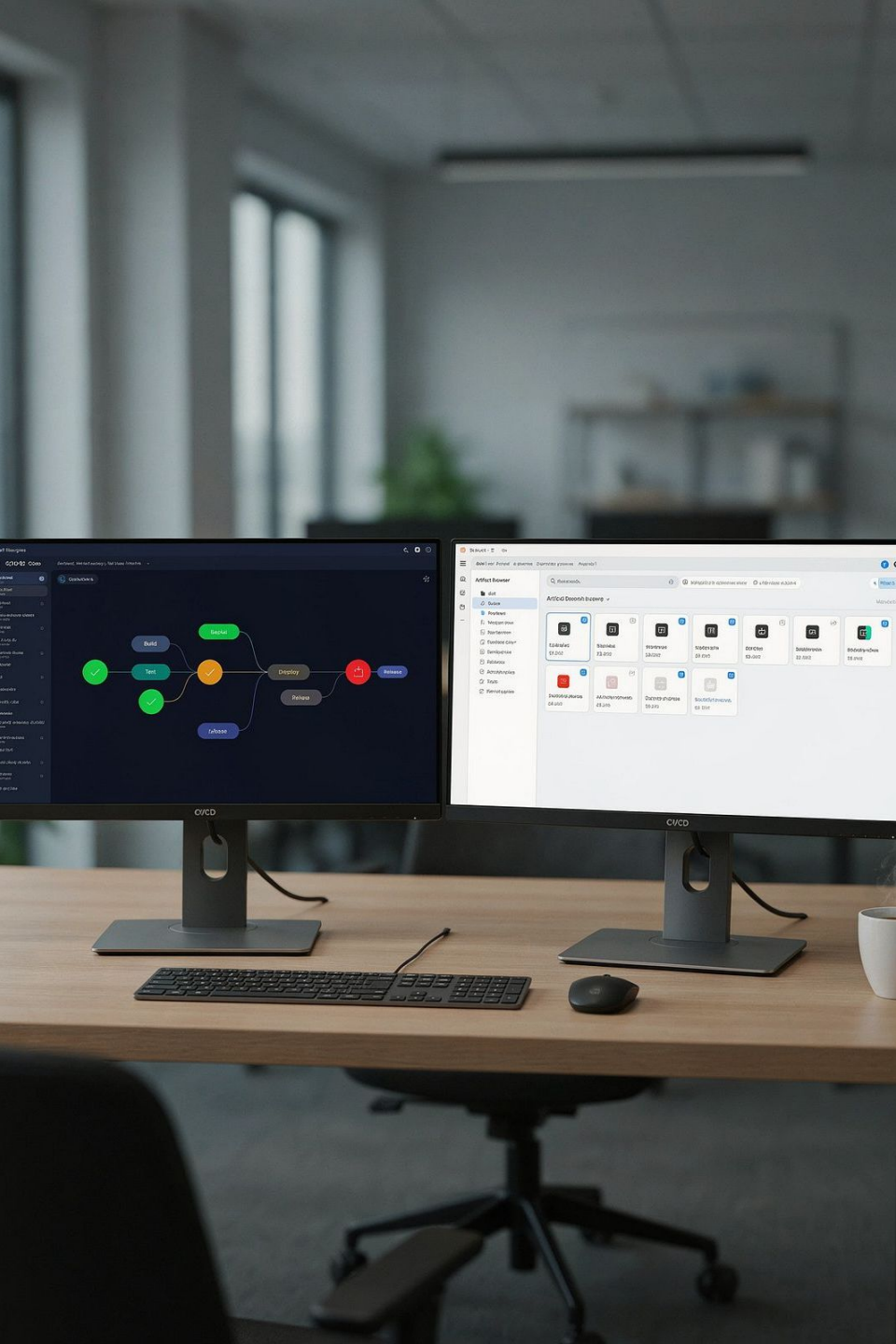
Our artifacts — produced by Jenkins builds

Remote

External artifacts — proxied through Artifactory

Virtual

Combined access point over multiple repositories



SECTION 3

Jenkins + Artifactory

Publishing an Artifact

CI Pipeline Flow



Before & After

Maven build output:

```
target/  
└── sample-app-1.0.jar
```

Published to Artifactory:

```
sample-app/  
└── 1.0/  
    └── sample-app-1.0.jar
```

Retrieving an Artifact

Deployment does not need to rebuild the application. The same tested artifact moves through environments.



DEV	QA
UAT	PROD

📄 **Build Once, Deploy Many** — the same artifact is promoted across all environments.

Build Once, Deploy Many

❌ Avoid — Rebuilding Per Environment

DEV → Build A

QA → Build B

UAT → Build C

PROD → Build D

Different builds introduce uncertainty and inconsistency.

✅ Preferred — Promote One Artifact

Source Code

↓ Build Once

↓ Artifact v1.0.105

↓ Artifactory

├── DEV

├── QA

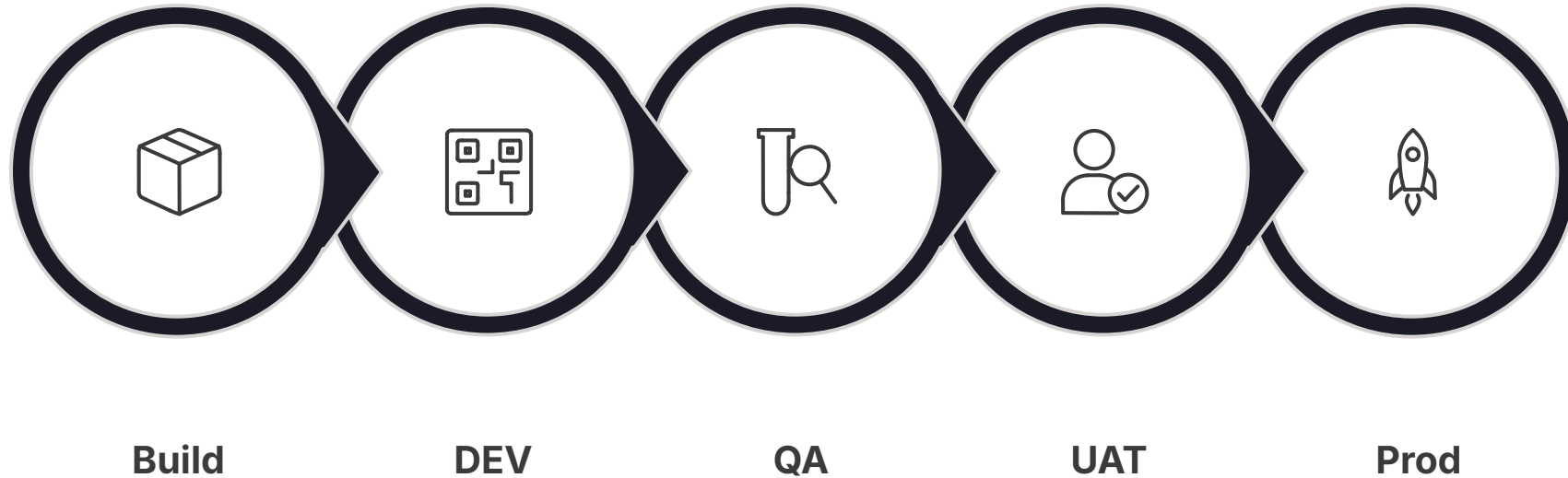
├── UAT

└── PROD

Promote the same validated artifact rather than rebuilding for every environment.

Artifact Promotion

An artifact progresses through the delivery lifecycle after validation at each stage.



Promotion Depends On

- Successful tests & quality checks
- Environment validation
- Approval & release readiness

Goal

Only validated artifacts should progress toward production.

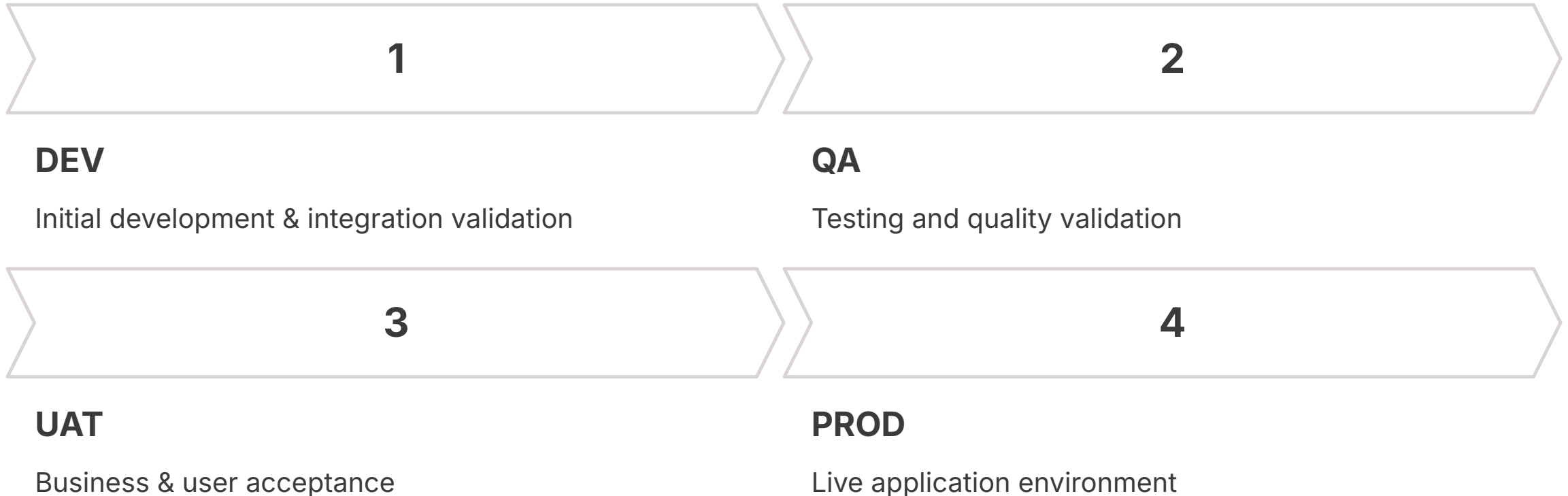


SECTION 4

Deployment Awareness

Deployment Across Environments

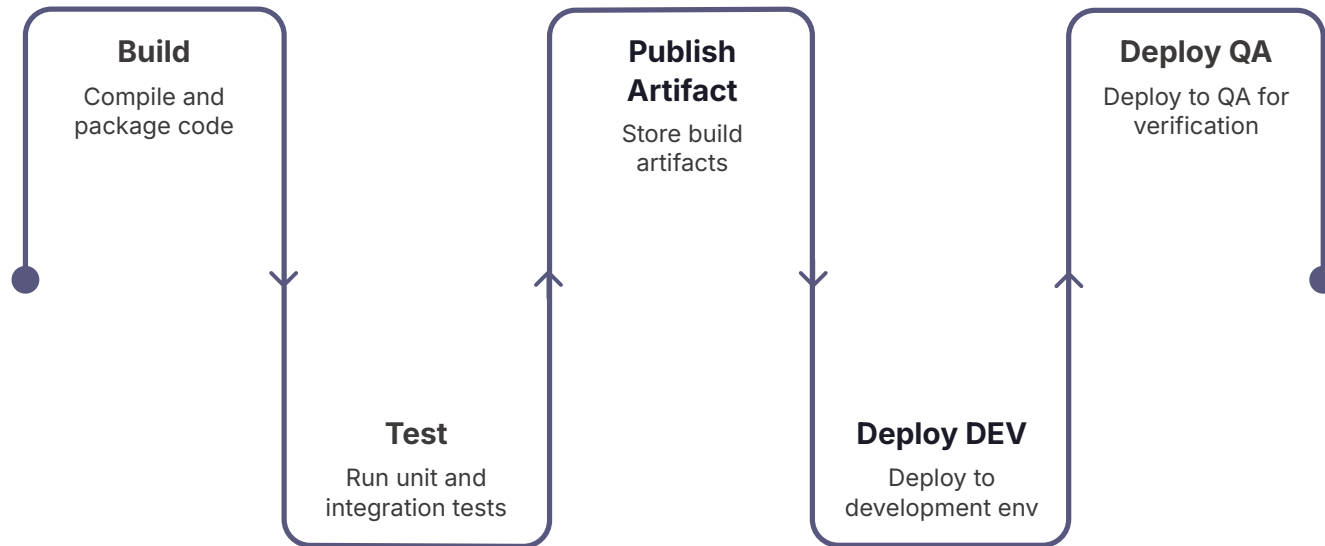
A typical enterprise delivery flow progresses through four environments, each with a distinct purpose.



CI/CD Pipelines control how software progresses between these environments.

Manual Approval Gates

Not every deployment should happen automatically. Approval gates provide a control point before sensitive deployment activities.



Jenkins Pipeline Example

```
stage('Approval') {  
  steps {  
    input message: 'Deploy to UAT?',  
           ok: 'Deploy'  
  }  
}
```

The pipeline pauses and waits for a human decision before proceeding.

Rollback Strategies

What if Deployment Fails?

Failure Detected

Monitor alerts and logs show critical errors.

Rollback Triggered

Initiate automated rollback procedure immediately.

New Version Deployed

Release sample-app-1.0.105.jar to production.

Previous Stable Restored

Redeploy sample-app-1.0.104.jar from Artifactory.

Artifactory Enables Rollback

```
sample-app-1.0.103.jar  
sample-app-1.0.104.jar  
sample-app-1.0.105.jar ← New
```

Previously validated artifacts remain available in Artifactory, making it straightforward to redeploy a known-good version when a rollback is required.

Day 4 Hands-On Lab

01

**Configure Jenkins
Credentials**

02

Build the Application

03

Publish to Artifactory

04

Retrieve the Artifact

05

Add Manual Approval

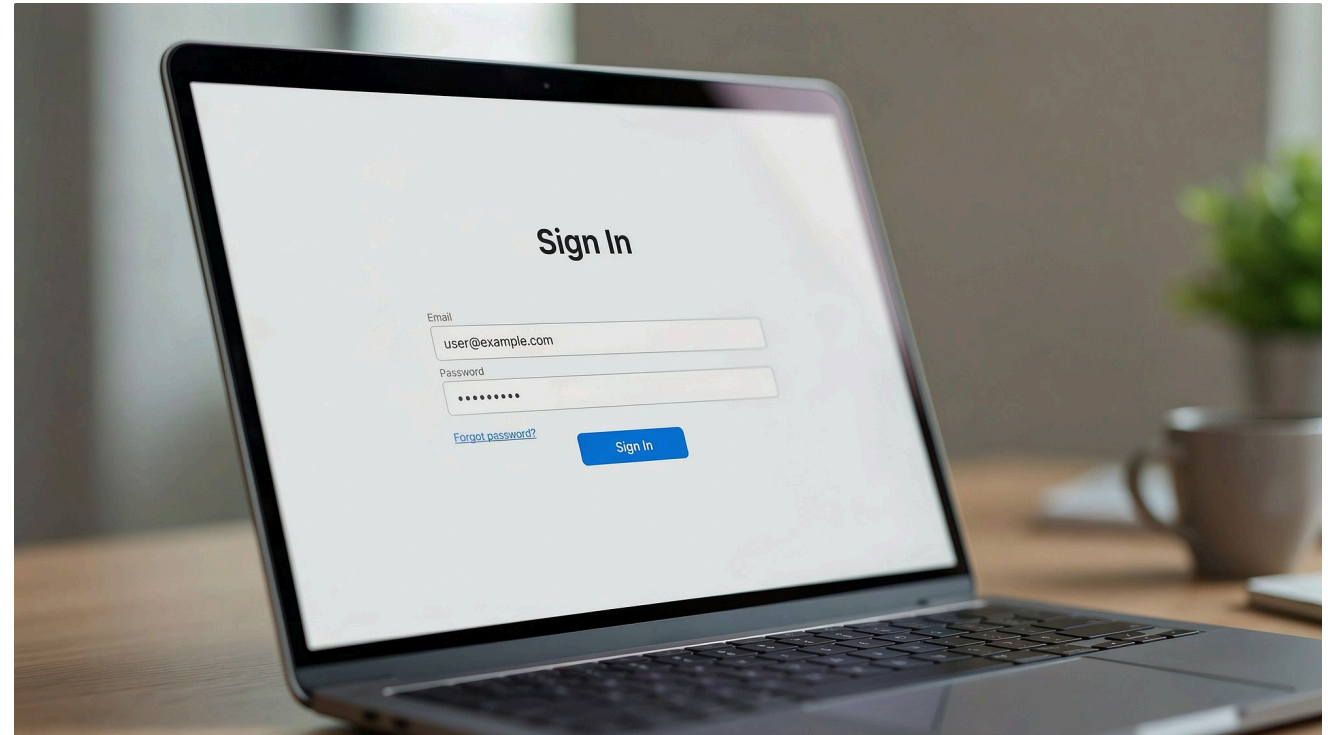
Lab 1 — Configure Jenkins Credentials



Use the **Credential ID** in your Pipeline so secrets never appear in source code.

Steps

1. Navigate to the Jenkins Credentials area
2. Create a credential for the lab Artifactory instance
3. Set the Credential ID to `artifactory-credentials`
4. Reference the credential from the Pipeline — not username/password directly in the Jenkinsfile



Lab 2 — Build the Application

Reuse the Maven application from Day 3.

```
stage('Build') {  
  steps {  
    sh 'mvn clean package'  
  }  
}
```

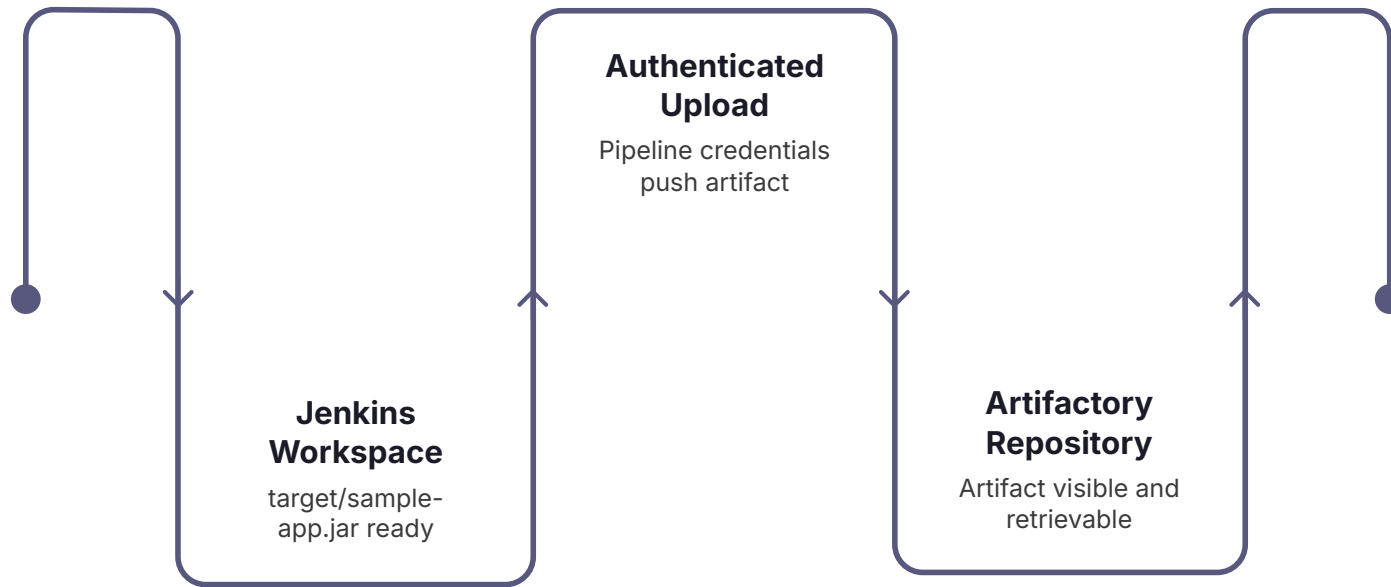
Expected Output

```
target/sample-app.jar
```

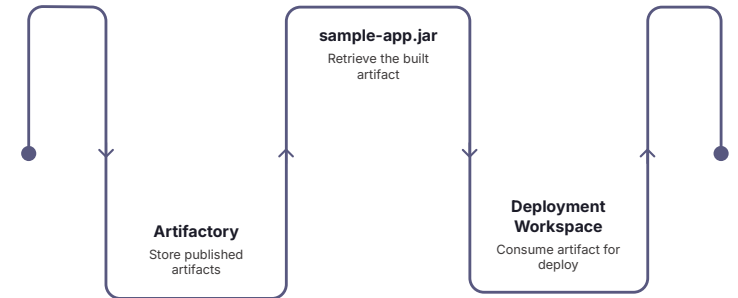
Confirm the artifact is produced in the workspace before moving to the publish step.

Labs 3 & 4 — Publish & Retrieve

Lab 3 — Publish to Artifactory



Lab 4 — Retrieve the Artifact



Verify the artifact can be consumed **without rebuilding** the source code.

Verify the artifact is visible in the Artifactory UI after the pipeline runs.

Lab 5 — Add Manual Approval

```
stage('Approval') {  
  steps {  
    input message: 'Proceed with deployment?',  
           ok: 'Deploy'  
  }  
}
```

Observe the Pipeline Pause

Deployment may remain **simulated** — the lab goal is deployment/operations awareness.

Build

Publish

 **Waiting for Approval**

Approve → Deployment